

Summary Report — Dungeon Grid Arena

Kerem Inci 德珍

2025080281

C++ Final Project

Source code: <https://github.com/xDKRMx/Dungeon-Grid-Arena.git>

1. Project overview

Dungeon Grid Arena is a turn-based, chess-inspired dungeon rogue-like game implemented from scratch in modern C++17. The goal of the project was to demonstrate every mandatory and optional knowledge point of the course (classes, file I/O, multi-file structure, inheritance and polymorphism, templates, hand-rolled linked lists) inside a single playable program instead of an isolated academic exercises.

The codebase has two different renderers behind a single `IRenderer` interface, but they are not equal:

- the **Raylib graphical renderer** is the canonical experience — full 1280×760 HUD, procedural sound effects, streamed OGG music, the Death Dungeon visual mode shift, and every other showcase feature; this is the build all screenshots and the demo video target.
- the **ASCII console renderer** is intentionally developed as a minimal smoke-test that runs the same game core through a terminal. Its purpose is to prove that the renderer abstraction is loosely coupled — it carries the gameplay logic (movement, combat, save / load, menus) but not the audio, the visual effects, or the Death Dungeon mode.

Both binaries are produced from the same C++17 source through a single compile flag (`-DDGA_WITH_RAYLIB`) which adds the Raylib renderer to the build. The game core never knows which renderer is active because every drawing and input operation crosses the abstract `IRenderer` interface.

Project size at submission:

Metric	Value
Source files (<code>.h</code> + <code>.cpp</code>)	51 files in 9 layered folders
Approximate lines of C++ code	~16 000
Test files	7 files (vendored doctest)
Test cases / total assertions	7 / 540 094 (all pass)
Audio assets	2 streamed OGG tracks
External libraries	Raylib 5.5 (graphical build only)

2. Goals and scope

The project targets **every** rubric item from the course description rather than picking a minimal subset:

- Class-driven design with file I/O.
- Multi-file structure: every non-trivial class has a `.h` + `.cpp` pair.
- Inheritance and polymorphism through three abstract bases (`Entity`, `Item`, `Ability`) and one pure interface (`IRenderer`).
- Templates: `Grid<T>` (2-D bounded buffer) and `RingBuffer<T>` (fixed capacity FIFO).
- Hand-implemented singly linked list (`EventLog`) used by the HUD.
- Arrays — configuration tables, audio frequency lookup tables, deterministic enemy-roll buckets.
- Library integration (Raylib) for graphics, audio synthesis, and streaming music.

The gameplay scope was chosen so that each of these points has a *reason* to exist in the running game:

- Inheritance models the real "is-a" relationships in the world (a Rook IS-A chess-style Enemy IS-A grid Entity).
- Templates are picked where the abstraction genuinely pays off (`Grid<T>` is shared between the dungeon map and the visited-cells buffer of the BFS).
- The hand-rolled linked list backs the on-screen event log so the player literally sees the data structure working every turn.
- File I/O is shown twice — a transactional save/load and a sorted top-N leaderboard — and demonstrates two different parsing styles.

3. Software architecture

3.1 Strict layered design

Every source folder under `src/` is one layer in the dependency graph, and each layer is allowed to include only headers from the layers strictly below it. Nothing in the lower layers ever includes a header from a higher layer. Here is the general layer design on a diagram:



The strictness is checked by simple `grep` on every commit: a header in `world` may not contain `#include "systems/..."`, and so on. This rule single-handedly prevents a common C++ trap (a tangle where A includes B and B includes A through a transitive chain).

3.2 Renderer abstraction

The `IRenderer` interface in `render/IRenderer.h` is a pure virtual class with four core methods (`drawFrame`, `pollInput`, `drawMenu`, `drawMessage`) plus optional hooks for transient effects (`showFireEffect`, `showNovaEffect`, ...) and audio (`showPickupSound`, `setBossMusicActive`, `setMasterMusicVolume`, ...). Every effect / audio hook has a default no-op body so the ASCII smoke-test renderer compiles unchanged.

The concrete `RaylibRenderer.cpp` is the only translation unit in the entire project that includes `raylib.h`. This means the rest of the codebase can be compiled into the test runner — and the ASCII smoke-test build — without Raylib installed at all.

3.3 No globals, single-source-of-truth

Every piece of run state lives inside one owned `GameState` instance which is constructed in `main.cpp` and passed by reference into every system. There are zero global variables and zero singletons. The deterministic random number generator (`Rng`) lives on `GameState` so every system that needs randomness draws from the same shared, seeded source — making the full run reproducible from a single seed.

4. Implementation challenges and solutions

This section covers the four most interesting problems I had to solve while building the project.

4.1 The forward-declaration knot between Entity, Player, Enemy, and GameState

`GameState` owns a `std::vector<std::unique_ptr<Enemy>>`. In a naive implementation the `GameState` header would include `Enemy.h`, which in turn includes `Entity.h`, which in turn references `Player`. As soon as `Player` or any other header indirectly needed `GameState`, the chain closed into a circular dependency that refused to compile.

The fix was to **forward-declare** `Enemy` in `GameState.h` and then move the `~GameState()` destructor body out-of-line into `GameState.cpp`, where the full `Enemy` type is finally included. A `unique_ptr<Enemy>` is legal to hold when `Enemy` is incomplete — but it cannot be *destroyed* until the deleter is instantiated, and the deleter is instantiated wherever the destructor of the owning class lives. By placing the destructor in a `.cpp` we control exactly which translation unit needs the complete type.

This is a small idiom but it took a full afternoon to discover the first time, and it now appears in `GameState`, `Player`, `Game`, and a couple of other places. The lesson I really learned from the project is that a `unique_ptr` to an incomplete type is fine if nothing in this header ever destroys one.

4.2 Marrying a frame-driven library to a blocking game loop

`IRenderer::pollInput()` is contractually blocking: the game core calls it, expects a single `InputCommand` back, and does nothing else until the player presses a key. This is the natural shape for a turn-based game.

Raylib, however, is a frame-driven library. `IsKeyPressed(KEY_W)` is only useful inside a frame loop that calls `BeginDrawing / EndDrawing` every iteration to pump the OS message queue. A blocking poll that did not refresh the screen would freeze the window.

The bridge is a **cached composition pattern** inside `RaylibRenderer`:

- Every `drawFrame / drawMenu / drawMessage` call stores its inputs into member fields (`cachedState_`, `menuOptions_`, `messageLines_`, ...).
- `renderCurrentScreen()` is a private helper that re-paints the cached composition through one `BeginDrawing / EndDrawing` pair.
- `pollInput()` runs an inner `while(true)` loop that calls `renderCurrentScreen()` on each iteration *until* a recognised key fires.

The result is a renderer that satisfies the blocking contract on the outside but redraws at 60 FPS internally, keeps the OS responsive, and lets the game core stay completely renderer-agnostic.

4.3 Transactional save and load

The naive approach to save/load is `state_.setX(...)` line by line as the parser reads the file. This silently corrupts the live state if the file is truncated halfway through, because half of the values have been overwritten and the other half still hold the pre-load values.

`SaveManager::load` therefore implements a staging-then-commit flow:

1. Every parsed value lands in a *local* staging variable inside `load`.
2. The function tracks a `bool found*` flag for every required field.
3. After the entire file is parsed, the flags are checked. If anything is missing, the function returns `false` *without writing a single byte* to `GameState`.
4. Only on a fully validated parse is the staged data committed to `state_`.

The same pattern handles backwards compatibility. When I added the `gold_` field for the post-wave Shop feature, the format gained an optional `GOLD N` line. Old save files do not contain it, so the staging variable defaults to 0 and is committed unconditionally — old saves load cleanly with an empty wallet, no migration required.

4.4 Per-wave streaming music with mutual exclusion and master volume

There are two streaming OGG tracks — one for normal waves and one for boss waves (every fifth wave). The two must never overlap, the procedural ambient pad needs to duck underneath whichever streamed track is playing, and the Settings menu should be able to adjust a master volume scalar live without a restart.

The solution lives in `RaylibRenderer`:

- Each track has its own `Music` handle plus a `Loaded` and `Playing_` flag pair.
- `setBossMusicActive(true)` and `setNormalMusicActive(true)` are mutually exclusive — each one stops the other before starting itself, and seeks the newly started track back to frame 0 so every wave starts at the top of the song.
- The procedural ambient pad's volume is dropped to a *ducked* level whenever either streamed track is playing and is restored to its full level only when both streams are silent.
- `setMasterMusicVolume(float)` clamps the input to `[0.0, 1.0]`, stores it on a member, and re-applies it through `SetMusicVolume(*, base * master)` to every loaded track immediately so the slider in the Settings menu is audible without any restart.

The music switching is also called explicitly from `Game::playingLoop` after every wave advance with an `stopBossMusic()` first to guarantee that two

consecutive *normal* waves still restart the song from the beginning, giving the player a fresh musical beat at every wave boundary.

4.5 Two iterative UX bug fixes worth mentioning

- **Game-Over freeze:** the original game-over screen used `pollInput`, which routes `F` into the fire direction prompt and `1` into the dash direction prompt — both inner loops that ignore `Space` and `Q`. A reflex tap on the wrong key after death made the screen *appear* frozen (the fire prompt was technically active but the player did not know how to escape it). The fix was to add a narrow `IRenderer::waitForAnyKey()` hook that listens only for `Space / Enter / Q / Esc`.
- **Two-panel wave-clear menu → single combined panel:** the first iteration of the post-wave flow showed a free-upgrade panel and then, separately, a paid-shop panel. The back-to-back Menus were confusing. The current build merges them into a single panel with section headers `[ FREE ]` and `[ PAID — gold N ]`, and the cursor automatically skips over rows the player cannot afford so unaffordable items are visible but not selectable.

5. Algorithms and data structures used

The project intentionally exercises a wide spread of algorithm and data structure ideas — every one of them is in service of a real gameplay feature, not an academic exhibit:

Algorithm / structure	Used by	Real role in gameplay
Breadth-first search	<code>Pathfinder::shortestPath</code>	Every enemy step toward the player.
Bresenham line traversal	<code>LineOfSight::lineCells</code>	Projectile rays, fire-aim preview, visibility queries for Blink.
Drunkard's walk + flood-fill	<code>MapGenerator</code>	Dungeon generation, validated by full connectivity check.
Fisher–Yates partial shuffle	<code>UpgradeSystem</code> , <code>Shop</code>	Drawing 3 distinct upgrade cards / shop items per wave.
Insertion sort	<code>ScoreBoard::loadTop</code>	Sorted top-N leaderboard on every append.
Hand-rolled singly linked list	<code>EventLog</code>	Rolling buffer of recent gameplay messages shown in the HUD.
Templates <code>`Grid<T>`</code>	<code>world/GridMap</code> , BFS visited-buffer	Generic 2-D bounded array.

Algorithm / structure	Used by	Real role in gameplay
Templates 'RingBuffer<T>'	(kept for the EventLog's earlier draft)	Fixed-capacity FIFO with O(1) push / drop.
Finite state machine	GameStateMachine	Top-level run phases (MainMenu / Playing / UpgradeDraft / GameOver).
Polymorphism via virtual calls	Entity, Item, Ability, IRenderer	The single most important OOP technique in the codebase.
Procedural audio synthesis	RaylibRenderer.cpp makeXxxSound helpers	Every SFX (no .wav files on disk).
AudioStream callback	RaylibRenderer.cpp audioStreamCallback	Real-time procedural ambient pad (4-voice A-minor chord + LFO).

6. Testing strategy

The test suite uses **doctest 2.4.11** vendored as a single header and deliberately favours **property-based** tests over fixture-based tests.

A property test states an *invariant* the system should always satisfy and then exercises the system on a *generated stream* of randomised inputs. With this approach 7 test cases produce **540 094 distinct assertions** across the run — roughly five orders of magnitude more coverage than the same files would deliver if they had hand-coded one input each.

Concrete examples:

- `test_pathfinder.cpp` / `shortest connected path`: pick 1 000 random `(start, goal)` pairs on a 1 000-tile maze; for each pair, assert that the BFS returned path length equals the Chebyshev distance (a stricter invariant than "connected", because no shorter path can possibly exist on an orthogonal grid).
- `test_line_of_sight.cpp` / `Bresenham symmetry`: for 10 000 random `(p, q)` pairs, assert that the cell list of `lineCells(p, q)` equals the reverse of `lineCells(q, p)`. Catches off-by-one errors in either direction at once.
- `test_mapgen_invariant.cpp` / `connected map`: generate 100 dungeons with random seeds; for each, flood-fill from one floor tile and assert that every other floor tile is reached. This is the central design contract of `MapGenerator`.

The result on the submission machine:

```
PS C:\c++ final project> .\test_runner.exe
[doctest] doctest version is "2.4.11"
[doctest] run with "--help" for options
=====
[doctest] test cases:      7 |      7 passed | 0 failed | 0 skipped
[doctest] assertions: 540094 | 540094 passed | 0 failed |
[doctest] Status: SUCCESS!
PS C:\c++ final project> |
```

Full methodology, run instructions, and result-judgment criteria are in `tests/README.md`.

7. AI usage report

In the development of this project, an AI coding assistant (used through Kiro IDE, which is built on top of Claude Opus and runs the model with full repository context) helped me determine the correct architecture and also write the code, preventing having heavy technical issues. I want to be transparent about *what* the AI helped with, *what* I drove myself, and *what I understand* in the resulting code.

7.1 Tasks completed with AI assistance

The following parts of the project benefitted from AI help:

- **Initial multi-file scaffolding:** the layered folder structure, the empty `.h` / `.cpp` skeletons, and the doxygen-style banner comment at the top of each file.
- **Boilerplate:** constructor member-initialiser lists, getter / setter quartets, default destructors, the doctest `main` setup.
- **First-draft documentation comments:** every function has a description and parameter list. The AI produced the first pass; I reviewed each one for accuracy and edited many of them.
- **Test scaffolding:** the property-based test setup including the random-seed iteration patterns inside each `TEST_CASE`.
- **Procedural sound synthesis:** the Wave / Sound construction helpers in `RaylibRenderer.cpp` (`makeBeepSound`, `makeNoiseAndTone`, `makeChord`, `makeTwoNoteChime`, `makeVillainLaugh`). I ensured whether parameters were appropriate enough until the sounds matched what I wanted.

7.2 Iterative improvements I drove

The AI did not invent this project. Without the decisions, design judgements, and playtest-driven debugging listed below, the submission

would not exist in its current form — at best it would be a basic ASCII demo with no audio, no Death Dungeon, no shop, and a confusing two-panel post-wave flow.

- **Project scope and feature set.** I chose the topic (chess-inspired dungeon rogue-like), the gameplay rules (wave system, every fifth wave is a boss, abilities, charge meter, post-wave upgrade draft, paid shop, gold currency), and the control scheme. The AI built what I specified — every gameplay-facing feature exists because I asked for it.
- **Architecture decisions.** The strict 9-layer split (`core` → `world` → ... → `render`), the `IRenderer` abstract interface, the zero-globals rule, the choice to ship two renderers behind one interface, the choice to have a transactional save / load — every one of these was my call. I rejected several alternative shapes the AI offered (e.g. a singleton `Game::instance()` pattern, inheritance instead of composition for `EventLog`).
- **Single-panel wave-clear menu.** The AI's first cut produced two back-to-back panels (free upgrade, then shop). When I playtested it I judged the back-to-back flow confusing, decided the right fix was a single combined panel with section headers and unaffordable-row skipping, and directed the refactor.
- **Game-Over freeze diagnosis.** I reproduced the freeze at the keyboard, traced through the renderer code in my head, identified that pressing F on the dead screen routed the user into the fire direction prompt — a UX trap the original `pollInput` design could not avoid. I decided the right fix was a narrow `IRenderer::waitForAnyKey()` hook that accepts only Space / Enter / Q / Esc, and directed the change across three callsites.
- **Per-wave HP scaling formula.** When playtesting revealed that late-wave enemies could not keep up with the player's upgrade draws, I specified the tier-based HP bonus formula $((\text{wave}-1) / 3 \text{ tiers}, +6 \text{ HP per tier for normal enemies, } +24 \text{ HP per tier for bosses})$ and asked for `Entity::boostMaxHealth` to be added.
- **Death Dungeon mode.** I designed the entire visual escalation: the 9.5 s / 19 s trigger thresholds, the four overlay layers (vignette, embers, banner, bloody-brick palette swap), the once-per-wave villain-laugh trigger, the re-pulsing of the banner. I tuned every parameter (ember count, vignette alpha, pulse rate, banner duration) through repeated playtesting until the mode shift felt right.
- **Decoupled Score and Gold.** The original draft fed Treasure value only into `score_`, which made the post-wave Shop pointless. I identified the bug, specified the parallel `gold_` counter on `GameState` with its own accessors, decided that

Treasure pickups should credit BOTH counters (so the leaderboard formula stays unchanged), and directed the threading of `state.addGold(...)` through the new `BonusGold` upgrade card and shop filler.

- **Shop catalogue and balance.** I designed the four buff items (Piercer Round, Quickstep, Blink Chain, Twin Strike) plus the always-affordable Coin Cache filler, set their costs and charge counts, and specified which subsystem each effect belongs in (CombatSystem for the projectile / melee buffs, AbilitySystem for Blink Chain, TurnManager for Quickstep).
- **Audio system architecture.** I specified that every SFX should be procedurally synthesised at startup (no `.wav` files on disk), decided that the per-wave music should stream from OGG files extracted from real songs, and chose the boss-vs-normal track swap rule.
- **Bug fixes from playtest.** I reproduced and root-caused dozens of UX bugs at the keyboard: piercer-shot LOS bypass, twin-strike damage doubling, blink-chain cooldown skip, quickstep extra-move turn skip, the "Resume Game" entry never appearing, the wave 6 not clearing because of stale dead enemies in the vector, the music not restarting on consecutive normal waves, the illegal-state-transition log line on Q-to-menu, the auto-save not firing for first-time loaders, and many more. For each one I diagnosed the root cause and decided the fix.
- **Code-quality bar.** I insisted that both builds and the test runner compile **warning-free** under `-Wall -Wextra`. Every time the AI introduced an unused parameter or shadowing variable I flagged it and made the AI add the appropriate `[[maybe_unused]]` attribute or rename the local. The clean warning state of the final repository is mine.
- **Removed AI-vibe complexity.** When I noticed `Game::resetRun` used a placement-new + manual destructor pattern I judged too exotic for an undergraduate C++ submission, I directed the refactor to plain `GameState::reset() / Player::reset() / Inventory::clear() / EventLog::clear()` methods.

7.3 Problems I solved during the AI iteration

- Several forward-declaration cycles the first draft created. Once I understood the pattern (a `unique_ptr<Incomplete>` member needs an out-of-line destructor in a TU that has the complete type) I applied it consistently across `GameState`, `Game`, `Player`, and so on.
- A save-load corruption risk in the first version of `SaveManager`, which wrote into `GameState` line-by-line as it parsed. I specified the staging-then-commit transactional pattern before the rewrite.

- An audio-device shutdown order issue: closing `InitAudioDevice` before stopping the streamed music produced an audible click and occasional hang on quit. I diagnosed this and specified the teardown order in the destructor.
- The two-panel-confusion, game-over-freeze, and wave-not-clearing UX bugs above.
- The "save file does not exist" trap, where Load Game would always fail because the player had to remember to press Ctrl+S first. I specified the auto-save on resetRun / wave advance / quit-to-menu policy that turned Load Game into a reliable feature.

7.4 My understanding of the code

- I can explain the BFS in `Pathfinder` line by line — frontier queue, visited buffer, parent map, path reconstruction.
- I can explain Bresenham in `LineOfSight::lineCells`, including why the symmetric version produces a reversed cell list when the endpoints are swapped.
- I can explain `Entity::boostMaxHealth`, the wave-scaled enemy HP, and how `applyAttack` decomposes damage into armour absorption first and HP afterwards (with twin-strike doubling layered on top before the decomposition).
- I can explain `SaveManager::load`'s staging fields and why **every** required tag has a `found*` flag — and why GOLD is the one optional field for backwards compatibility.
- I can explain why `IRenderer` has a default no-op for every audio / effect hook (so the ASCII smoke-test build inherits a working stub without having to override).
- I can demonstrate the running test suite and read its property invariants out of the source.

7.5 Honest assessment of AI reliance

The AI was a force multiplier for *speed* and for *boilerplate*. The *design* decisions — the layering, the `IRenderer` interface, the transactional save, the single-panel wave-clear menu, the per-wave HP scaling, the Death Dungeon visual escalation, the decoupling of Gold from Score — came from my own iteration on top of what the AI produced.

8. Achievements and lessons learned

- I delivered a complete, polished C++17 game with two fully swappable renderers behind a single abstract interface, exercising every optional rubric item the course asked for.

- I learned the practical idioms of large C++ projects: forward-declaration hygiene, out-of-line destructors for incomplete-type ownership, strict layering enforced by include rules, and `unique_ptr` ownership transfer via `std::move`.
- I learned that **transactional load** is the only safe load — parse fully into local staging, then commit, never the other way around.
- I learned how a frame-driven library (Raylib) can be wrapped in a blocking interface (`IRenderer::pollInput`) by introducing a cached composition and an internal frame loop.
- I learned that **property-based tests** with random seeds catch entire classes of bugs — symmetry violations, off-by-one indexing, broken invariants — that a hand-coded fixture test would simply miss because it never tries the failing input.
- I learned how to keep a build warning-free with `-Wall -Wextra` even while a codebase grows past 50 files, by treating every warning as a bug and fixing it the moment it appears.

9. Future work

If I had another two weeks I would:

- Split `RaylibRenderer.cpp` (currently the largest source file in the project) into separate translation units for the synth helpers, the HUD, and the effect rendering. The current monolithic file is heavily commented but still oversized.
- Add a separate **SFX volume slider** on top of the existing master music slider, persisted to a small `data/settings.cfg`.
- Implement a small in-game tutorial overlay for first-time players (the current onboarding relies entirely on the controls strip and the user manual).
- Add network co-op (each player owns a `Player` instance, the server authoritative `GameState` is replicated). The renderer abstraction already cleanly hides which player is "you", which would make the client side feasible without a major rewrite.

10. Conclusion

Dungeon Grid Arena hits the C++ Programming course's full rubric and delivers a real playable game on top of it. Its architecture is layered, its data structures are real (a hand-rolled linked list does back the HUD's event log; templates do underpin the dungeon grid), and its file I/O is two genuinely different stories (transactional save/load and a sorted leaderboard). The project taught me concrete, reusable C++17

patterns I will carry into future work, and the AI assistance made it possible to complete the breadth of features within the timeframe — but the design and the polish are mine.

Appendix A — File / line counts

The following figures are produced by running the test runner build and the `Get-ChildItem` line counts on the submission tree. Numbers will vary by $\pm$ a few hundred lines if the project is rebuilt with a different working tree.

Metric	Value
Source files (<code>.h</code> + <code>.cpp</code>)	51
Approximate total lines of C++ code	~16 000
Lines of commentary / doc-comments	~6 000
Test cases	7
Total assertions in the test suite	540 094
Build commands (warning-clean)	3

Appendix B — Sample event log output (game)

```
PS C:\c++ final project> .\game_raylib.exe
INFO: Initializing raylib 5.5
INFO: Platform backend: DESKTOP (GLFW)
INFO: Supported raylib modules:
INFO:   > rcore:..... loaded (mandatory)
INFO:   > rlgl:..... loaded (mandatory)
INFO:   > rshapes:... loaded (optional)
INFO:   > rtextures.. loaded (optional)
INFO:   > rtext:..... loaded (optional)
INFO:   > rmodels... loaded (optional)
INFO:   > raudio:.... loaded (optional)
INFO: DISPLAY: Device initialized successfully
INFO:   > Display size: 2560 x 1440
INFO:   > Screen size: 1280 x 760
INFO:   > Render size: 1280 x 760
INFO:   > Viewport offsets: 0, 0
INFO: GLAD: OpenGL extensions loaded successfully
INFO: GL: Supported extensions count: 403
INFO: GL: OpenGL device information:
```

Appendix C — Test runner output (canonical PASS)

```
• PS C:\c++ final project> .\test_runner.exe
[doctest] doctest version is "2.4.11"
[doctest] run with "--help" for options
=====
[doctest] test cases:      7 |      7 passed | 0 failed | 0 skipped
[doctest] assertions: 540094 | 540094 passed | 0 failed |
[doctest] Status: SUCCESS!
○ PS C:\c++ final project> █
```